

中山大學
SUN YAT-SEN UNIVERSITY

基于表达式的计算器 ExprEval

实验 4-设计文档

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 李文军

起 始 日 期: 2026 年 4 月

结 束 日 期: 2026 年 5 月

实 验 地 点: B203

目录

1 实验简述 **3**
1.1 实验目的 . . . 3
1.2 实验内容 . . . 3
1.3 开发环境 . . . 3

2 分析与设计 **4**
2.1 BNF 语法的二义性分析 . . . 4
2.1.1 运算符优先级的二义性 . . . 4
2.1.2 一元取负与二元减法的重载二义性 . . . 4
2.1.3 三元运算符的结构二义性 . . . 4
2.1.4 二义性的解析方案 . . . 4
2.2 整体架构 . . . 5
2.3 异常体系设计 . . . 5
2.4 词法分析器设计：Scanner.java . . . 6
2.4.1 有限状态自动机（DFA）总览 . . . 6
2.4.2 数值扫描 DFA . . . 6
2.4.3 标识符/关键字 DFA . . . 7
2.4.4 运算符识别 . . . 8
2.4.5 Token 类型枚举 . . . 8
2.5 语法分析器设计：Parser.java . . . 8
2.5.1 算符优先分析（OPP）核心思想 . . . 8
2.5.2 优先关系与优先级层次 . . . 9
2.5.3 哨兵机制 . . . 9
2.5.4 语义动作与类型检查 . . . 9
2.5.5 预定义函数的实现 . . . 10
2.6 错误处理覆盖 . . . 10

3 核心代码 **11**
3.1 Scanner.java ——DFA 数值扫描 . . . 11
3.2 Parser.java ——算符优先分析关键方法 . . . 11
3.2.1 优先关系比较 . . . 11
3.2.2 归约语义动作 . . . 12
3.2.3 括号与函数调用处理 . . . 12

4 结果分析 **13**
4.1 编译结果 . . . 13
4.2 测试结果总览 . . . 13
4.3 简单与标准测试集验证 . . . 14
4.4 自定义强化测试验证 . . . 15
4.5 GUI 界面验证 . . . 17
4.5.1 GUI 正常求值 . . . 17
4.5.2 GUI 词法错误拦截 . . . 17

4.5.3 GUI 语法错误拦截 18
4.5.4 GUI 语义错误拦截 18

5 实验总结 **18**
5.1 技术收获 18
5.2 遇到的难点与解决方案 19
5.3 实验总结 19

1 实验简述

1.1 实验目的

本实验旨在实现一个基于表达式的科学计算器 **ExprEval**。与实验三的后缀表达式转换器相比，ExprEval 不仅需要将中缀表达式转换为后缀形式，还要求对表达式进行完整的词法分析、语法分析以及语义求值，最终返回计算结果。

通过本实验，能够深入理解编译原理中以下核心概念：

1. **词法分析（Lexical Analysis）**：使用确定性有限状态自动机（DFA）将字符流转换为 Token 流；
2. **语法分析（Syntax Analysis）**：使用算符优先分析（Operator Precedence Parsing, OPP）将 Token 流构建为语法树并同步求值；
3. **语义分析（Semantic Analysis）**：在归约过程中进行类型检查和错误检测；
4. **异常处理**：设计完整的三层异常体系，精确报告各种词法、语法和语义错误。

1.2 实验内容

实现一个完整的表达式计算器，支持以下特性：

- **数值类型**：整数、小数、科学记数法（如 `1.2e-3`、`2.25E+2`）
- **布尔常量**：`true`、`false`
- **算术运算符**：`+`、`-`（含一元取负）、`*`、`/`、`^`
- **关系运算符**：`<`、`>`、`<=`、`>=`、`==`、`!=`
- **逻辑/位运算符**：`&`、`|`、`!`、`~`
- **三元条件运算符**：`?:`
- **预定义函数**：`sin`、`cos`、`tan`、`log`、`sqrt`、`abs`、`pow`、`max`、`min`（共 9 个）
- **括号嵌套**：支持任意深度的括号表达式

1.3 开发环境

- 操作系统：Windows 11
- 语言与工具：Java（JDK 1.5+ 兼容）+ VS Code
- 构建：批处理脚本 `build.bat`
- 测试框架：XML 驱动的自动化回归测试
- 文档：LaTeX + `design.tex`

2 分析与设计

2.1 BNF 语法的二义性分析

实验手册第 2.3.1 小节以标准 BNF 定义了输入表达式的形式化语法。经分析，该语法定义**存在二义性**，主要体现在以下三个方面。

2.1.1 运算符优先级的二义性

BNF 产生式是纯语法的，并未编码运算符优先级信息。例如产生式：

```
ArithExpr → ArithExpr + ArithExpr | ArithExpr * ArithExpr
```

对于输入 `1 + 2 * 3`，可以推导出两棵不同的语法树：

- 先将 `1 + 2` 归约为 `ArithExpr`，再与 `* 3` 结合 → 结果为 9
- 先将 `2 * 3` 归约为 `ArithExpr`，再与 `1 +` 结合 → 结果为 7

这是典型的" 运算符优先级歧义"——BNF 自身无法区分 `+` 与 `*` 的执行次序。

2.1.2 一元取负与二元减法的重载二义性

BNF 中同时存在两条以 `-` 为核心的产生式：

- `ArithExpr → -ArithExpr`（一元取负，优先级 9）
- `ArithExpr → ArithExpr -ArithExpr`（二元减法，优先级 6）

对于输入 `2 - 3 * -4`，扫描到第二个 `-` 时，仅凭 BNF 无法判断应展开为一元取负还是二元减法。

2.1.3 三元运算符的结构二义性

类似经典的" 悬空 else" 问题，BNF 中的三元产生式：

```
ArithExpr → BoolExpr ? ArithExpr : ArithExpr
```

对于嵌套输入如 `a ? b ? c : d : e`，存在两种解释：

- `a ? (b ? c : d) : e`（右结合，与 C 语言一致）
- `(a ? b) : (c ? d : e)`（非标准结合）

2.1.4 二义性的解析方案

本实验采用**算符优先分析（OPP）**技术消解上述所有二义性：

1. **优先级驱动**：通过 10 级优先关系表（参见表 2），`*` 优先级（7）高于 `+`（6），比较优先级时自动归约高优先级算符，从而保证 `1 + 2 * 3` 先算乘法。
2. **上下文判别一元/二元减**：Scanner 阶段统一标记 `-` 为 `TokenType.SUB`。Parser 阶段通过布尔标志" 前驱是操作数" 判定：若前一 Token 为空/运算符/左括号，则识别为 `NEG`（一元取负）；否则为 `"-"`（二元减法）。

3. **结合性规则**：优先关系表的结合性列明确标注右结合算符（如^、?:、NEG）。当栈顶与输入算符优先级相等时，右结合则移进，左结合则归约，从而将 `a ? b ? c : d : e` 解析为右结合形式。

通过上述三种机制的组合，OPP 在不修改原始 BNF 的前提下高效消解了所有二义性，同时保证了分析器只需单遍扫描即可完成语法分析和语义求值。

2.2 整体架构

ExprEval 采用经典的" 词法 → 语法 → 语义" 三阶段流水线架构：

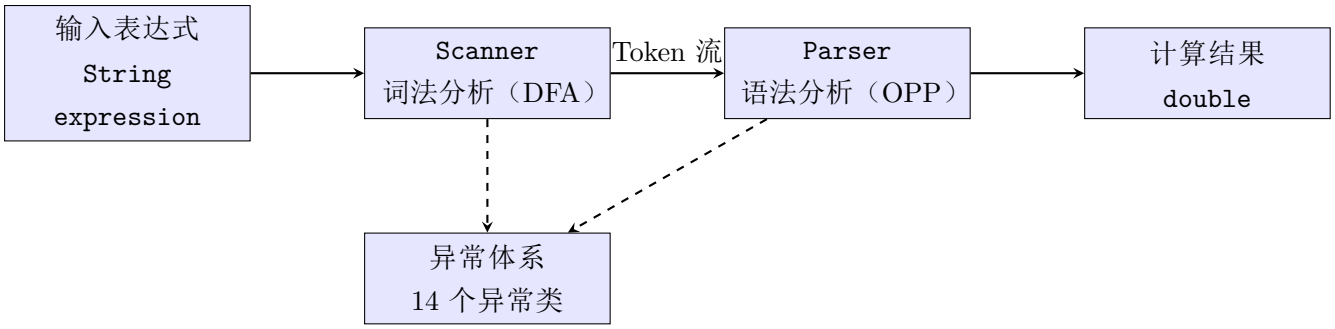


图 1: ExprEval 系统架构

调用入口 `Calculator.calculate(String)` 串联整个流程：

1. 构造 `Scanner`，对输入字符串进行词法分析；
2. 构造 `Parser`，使用 Scanner 提供的 Token 流进行算符优先分析；
3. Parser 在归约过程中同步求值，返回最终结果（double）。

2.3 异常体系设计

采用三层异常层次结构，自顶向下分别为通用的 `ExpressionException`、三大类别中间异常、以及具体异常子类：

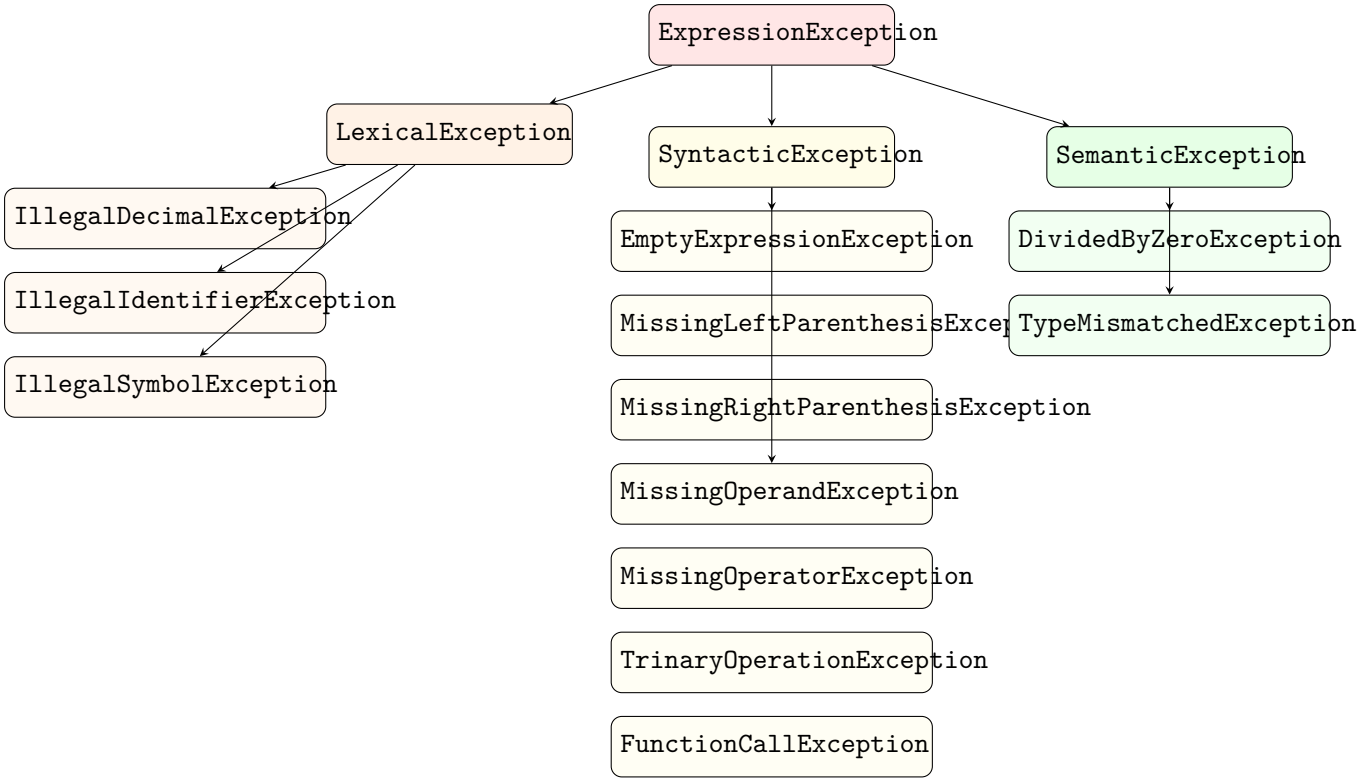


图 2: 异常类层次结构（共 16 个类）

2.4 词法分析器设计：Scanner.java

2.4.1 有限状态自动机（DFA）总览

Scanner 使用确定性有限状态自动机实现，核心入口方法 `nextToken()` 根据当前字符分发到不同的子 DFA：

- **数字开头**（`0-9`）→ 数值扫描 DFA（`scanNumber()`）
- **字母/下划线开头** → 标识符/关键字 DFA（`scanIdentifierOrKeyword()`）
- **运算符/分隔符首字符** → 单字符/双字符直接识别

2.4.2 数值扫描 DFA

数值扫描是 Scanner 中最复杂的状态机，支持整数、小数和科学记数法三种格式。共定义 6 个状态：

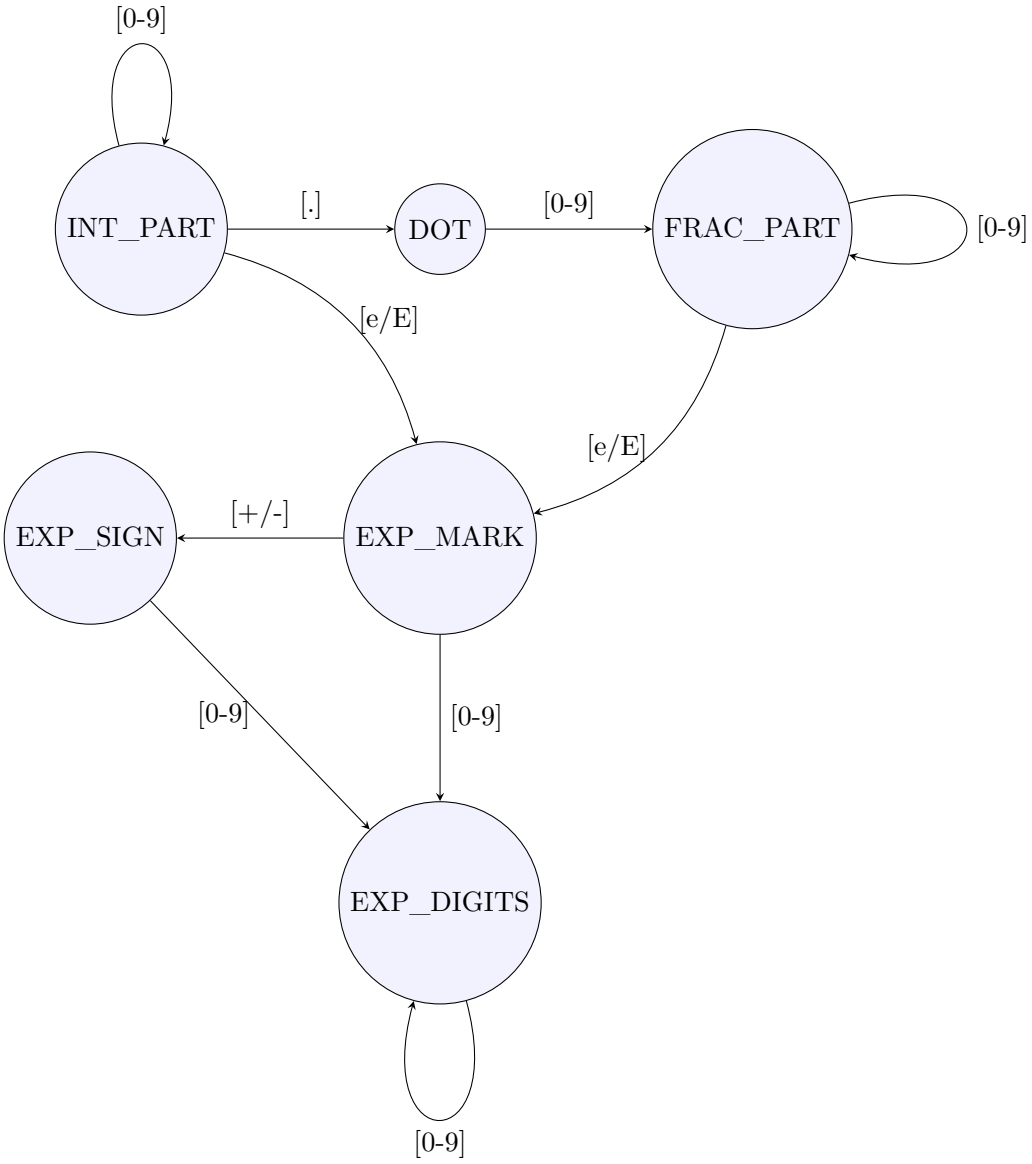


图 3: 数值扫描 DFA 状态转换图（NumberState 枚举）

敏感规则：在 DOT 状态（小数点后）及 EXP_SIGN 状态（指数符号后）必须至少有一位数字，否则抛出 `IllegalDecimalException`。例如 `10.E+5` 是非法的。

2.4.3 标识符/关键字 DFA

1. 扫描完整标识符（字母、数字、下划线）作为词素
2. 转化为小写后查表：
   - `true` / `false` → `TokenType.BOOLEAN`
   - 属于预定义函数集合（`sin`, `cos`, `tan`, `log`, `sqrt`, `abs`, `pow`, `max`, `min`）→ `TokenType.IDENTIFIER`
   - 其他 → 抛出 `IllegalIdentifierException`（如 `mix`）

2.4.4 运算符识别

一元取负（-）与二元减法在词法阶段统一标记为 `TokenType.SUB`，由 Parser 根据前驱 Token 上下文区分：若前驱为空、为运算符、或为 `(` `,` `?` `:`，则判定为一元取负。

双字符运算符（`<=` `>=` `==` `!=`）在单字符扫描基础上检查下一字符完成匹配。孤立的 `=` 抛出 `IllegalSymbolException`。

2.4.5 Token 类型枚举

Scanner 定义了 22 种 TokenType，涵盖本实验所有语法元素：

表 1: TokenType 枚举一览

类别	枚举值	示例
字面量	NUMBER, BOOLEAN, IDENTIFIER	42, true, sin
算术	PLUS(+), SUB(-), MUL(*), DIV(/), POW(^)	
关系	LT(<), GT(>), LE(<=), GE(>=), EQ(==), NE(!=)	
逻辑/位	AND(&), OR(|), NOT(!), BIT_NOT(~)	
三元	QUEST(?), COLON(:)	
分隔	LPAREN, RPAREN, COMMA	
结束	EOF	

2.5 语法分析器设计：Parser.java

2.5.1 算符优先分析（OPP）核心思想

Parser 采用双栈架构的算符优先分析算法：

- **算符栈**（`Stack<String>`）：存储运算符标识字符串（如 `"+"`、`"NEG"`、`"?:"`、`"("`、`"FUNC:sin"`）
- **值栈**（`Stack<Object>`）：存储中间结果（`Double`、`Boolean`）及哨兵标记

算法主循环：

```
while (true) {
    Token token = scanner.peek();
    switch (token.type) {
        case NUMBER/BOOLEAN: 压入值栈; break;
        case LPAREN:         压入 "(" 和哨兵; break;
        case RPAREN:         归约至 "(" 或 "FUNC:*"; break;
        case COMMA:          归约至 "FUNC:*"; break;
        case 运算符:         按优先关系归约或移进; break;
        case EOF:            归约全部剩馀算符; return 结果;
    }
}
```

Listing 1: OPP 主循环（简化伪代码）

2.5.2 优先关系与优先级层次

共定义 10 个优先级层次（数值越大优先级越高）：

表 2: 算符优先级表

优先级	算符	说明	结合性
0	#	栈底哨兵	—
1	?:	三元条件	右结合
2	?	三元条件前半（中间态）	右结合
3	&, |	逻辑与/或	左结合
4	!, ~	逻辑非/按位非	右结合
5	< > <= >= == !=	关系比较	左结合
6	+ -	加减	左结合
7	* /	乘除	左结合
8	^	幂	右结合
9	NEG	一元取负	右结合
MAX	(, FUNC:*	障碍符	—

优先关系比较规则：若栈顶优先级 $<$ 输入优先级，则移进；若栈顶优先级 $>$ 输入优先级，则归约；若相等，根据结合性决定（右结合则移进，左结合则归约）。

2.5.3 哨兵机制

值栈上插入特殊的哨兵对象（`private static final Object 哨兵 = new Object()`），用于标记括号子表达式和函数参数列表的边界：

- 遇到 (时：算符栈压入 "("，值栈压入哨兵
- 遇到函数调用时：算符栈压入 "FUNC: 函数名"，值栈压入哨兵
- 遇到) 且栈顶为 "("：归约哨兵之上的所有结果，弹出哨兵，将结果压回
- 遇到) 且栈顶为 "FUNC:*"：收集哨兵之上的参数列表，执行函数求值

2.5.4 语义动作与类型检查

每次归约时同步执行语义动作：

- **算术归约**：弹出两个 Double 操作数，计算结果，压回 Double
- **关系归约**：对 < > <= >= 弹出两个 Double，计算结果 Boolean；对 == != 支持同类型比较（Double vs Double 或 Boolean vs Boolean）
- **逻辑归约**：弹出两个 Boolean 操作数，计算结果 Boolean
- **一元归约**：弹出操作数进行 NEG（取负）、! 或 ~（取反）
- **三元归约**：弹出假分支、真分支、条件；条件为 true 保留真分支，否则保留假分支

类型不匹配时抛出 TypeMismatchedException。

2.5.5 预定义函数的实现

9 个预定义函数各自由 Java 标准库 `java.lang.Math` 提供计算支持：

表 3: 预定义函数实现映射

函数	参数个数	对应实现
`sin`	1	`Math.sin(a)`
`cos`	1	`Math.cos(a)`
`tan`	1	`Math.tan(a)`
`log`	1	`Math.log(a)`（参数必须为正）
`sqrt`	1	`Math.sqrt(a)`（参数必须非负）
`abs`	1	`Math.abs(a)`
`pow`	2	`Math.pow(a, b)`
`max`	≥ 2	`Math.max` 遍历
`min`	≥ 2	`Math.min` 遍历

参数个数不匹配时抛出 `FunctionCallException`,参数为非数值类型时抛出 `TypeMismatchedException`。

2.6 错误处理覆盖

ExprEval 能够检测并精确报告 10 类错误：

表 4: 错误覆盖矩阵

错误场景	抛出异常	测试用例 ID
非法数值格式（如 `10.E+5`）	`IllegalDecimalException`	E007
未识别标识符（如 `mix`）	`IllegalIdentifierException`	E008
非法符号（如 `@`、孤立 `=`）	`IllegalSymbolException`	
缺少左括号	`MissingLeftParenthesisException`	E001
缺少右括号	`MissingRightParenthesisException`	E002
缺少运算符	`MissingOperatorException`	E003, E012
缺少操作数	`MissingOperandException`	E004, E010, E014, E018
除零	`DividedByZeroException`	E005
类型不匹配	`TypeMismatchedException`	E006, E015–E017, E019
函数参数个数错误	`FunctionCallException`	E009, E013
三元运算符不完整（如 `1?2`）	`TrinaryOperationException`	E011
表达式为空	`EmptyExpressionException`	E020

3 核心代码

3.1 Scanner.java ——DFA 数值扫描

数值扫描是 Scanner 最核心的 DFA，以下是其完整实现：

```
private Token scanNumber() throws IllegalDecimalException {
    StringBuilder sb = new StringBuilder();
    NumberState state = NumberState.INT_PART;
    while (true) {
        switch (state) {
            case INT_PART:
                if (ch >= '0' && ch <= '9') {
                    sb.append(ch); advanceChar();
                } else if (ch == '.') {
                    sb.append(ch); advanceChar();
                    state = NumberState.DOT;
                } else if (ch == 'e' || ch == 'E') {
                    sb.append(ch); advanceChar();
                    state = NumberState.EXP_MARK;
                } else {
                    return new Token(TokenType.NUMBER,
                        sb.toString(),
                        Double.parseDouble(sb.toString()));
                }
                break;
            case DOT:
                // 小数点后必须至少有一位数字
                if (ch >= '0' && ch <= '9') {
                    sb.append(ch); advanceChar();
                    state = NumberState.FRAC_PART;
                } else {
                    throw new IllegalDecimalException(...);
                }
                break;
            // ... FRAC_PART, EXP_MARK, EXP_SIGN, EXP_DIGITS
        }
    }
}
```

Listing 2: 数值扫描 DFA（scanNumber 方法）

3.2 Parser.java ——算符优先分析关键方法

3.2.1 优先关系比较

```
private char 比较优先级(String 栈顶算符, String 输入算符) {
    int 栈顶优先 = 优先级(栈顶算符);
    int 输入优先 = 优先级(输入算符);

```

```
    if (栈顶优先 < 输入优先) return '<'; // 移进
    if (栈顶优先 > 输入优先) return '>'; // 归约

    // 优先级相等时，根据结合性决定
    if (是右结合(输入算符)) return '<'; // 右结合：移进
    else return '>';                    // 左结合：归约
}
```

Listing 3: 优先级与结合性判断

3.2.2 归约语义动作

```
private void 归约二元(String 算符) throws ExpressionException {
    Object 右 = 弹出操作数(算符 + " 的右操作数");
    Object 左 = 弹出操作数(算符 + " 的左操作数");

    if (是算术算符(算符)) {
        double 左值 = 要求数值(左, 算符);
        double 右值 = 要求数值(右, 算符);
        // +, -, *, / (检查除零), ^
        值栈.push(Double.valueOf(计算结果));
    } else if (是关系算符(算符)) {
        // <, >, <=, >=, ==, !=
        // ==/!= 支持布尔 vs 布尔
        值栈.push(Boolean.valueOf(比较结果));
    } else if (是逻辑算符(算符)) {
        boolean 左值 = 要求布尔(左, 算符);
        boolean 右值 = 要求布尔(右, 算符);
        值栈.push(Boolean.valueOf(左值 && 右值));
    }
}
```

Listing 4: 二元算符归约（简化）

3.2.3 括号与函数调用处理

```
private void 处理右括号() throws ExpressionException {
    归约至障碍符();
    String 栈顶 = 算符栈.peek();
    if (栈顶.equals("(")) {
        // 普通括号：收集哨兵之上的结果，弹出哨兵，压回结果
        算符栈.pop();
        Object 括号结果 = 收集哨兵之上的唯.一.结.果();
        弹出哨兵(")");
        if (括号结果 != null) 值栈.push(括号结果);
        scanner.advance();
    } else if (栈顶.startsWith("FUNC:")) {
        // 函数调用：收集参数列表并求值
        执行函数调用();
```

```
        scanner.advance();
    } else {
        throw new MissingLeftParenthesisException(...);
    }
    前驱是操作数 = true;
}
```

Listing 5: 右括号处理——区分普通括号与函数调用

4 结果分析

4.1 编译结果

运行 `build.bat` 编译成功（exit code = 0），Scanner.java、Parser.java、Calculator.java 及 14 个异常类全部编译通过，生成对应的 `.class` 文件。

```
(base) PS E:\3.2\Compilers\LAB\my_lab\lab4> dir bin\parser


    Directory: E:\3.2\Compilers\LAB\my_lab\lab4\bin\parser


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         2026/5/17     20:11            494 Calculator.class
-a----         2026/5/17     20:11           1681 Parser$1.class
-a----         2026/5/17     20:11          15367 Parser.class
-a----         2026/5/17     20:11           1292 Scanner$NumberState.class
-a----         2026/5/17     20:11           1607 Scanner$Token.class
-a----         2026/5/17     20:11           2210 Scanner$TokenType.class
-a----         2026/5/17     20:11           6157 Scanner.class
```

图 4: 编译成功输出（build.bat）

如图 4 所示，所有源文件零错误通过编译，为后续的测试与验证提供了可靠的构建基础。

4.2 测试结果总览

使用三套 XML 驱动的自动化测试用例集进行回归验证：

表 5: 测试结果汇总

测试集	正确用例	错误用例	总数	通过率
`simple.xml`	3	5	8	100%
`standard.xml`	6	10	16	100%
`mytest.xml`	10	10	20	100%
总计	**19**	**25**	**44**	**100%**

由表 5 可见，全部 44 个测试用例——涵盖正确表达式求值（19 个）与错误报告验证（25 个）

——均以 100% 通过率通过，证明了词法分析、语法分析和语义处理三个阶段的正确性与鲁棒性。以下各节将结合测试截图逐一展开分析。

4.3 简单与标准测试集验证

官方提供的 `simple.xml`（8 用例）和 `standard.xml`（16 用例）是实验软装置的基准回归测试。其运行结果如下：

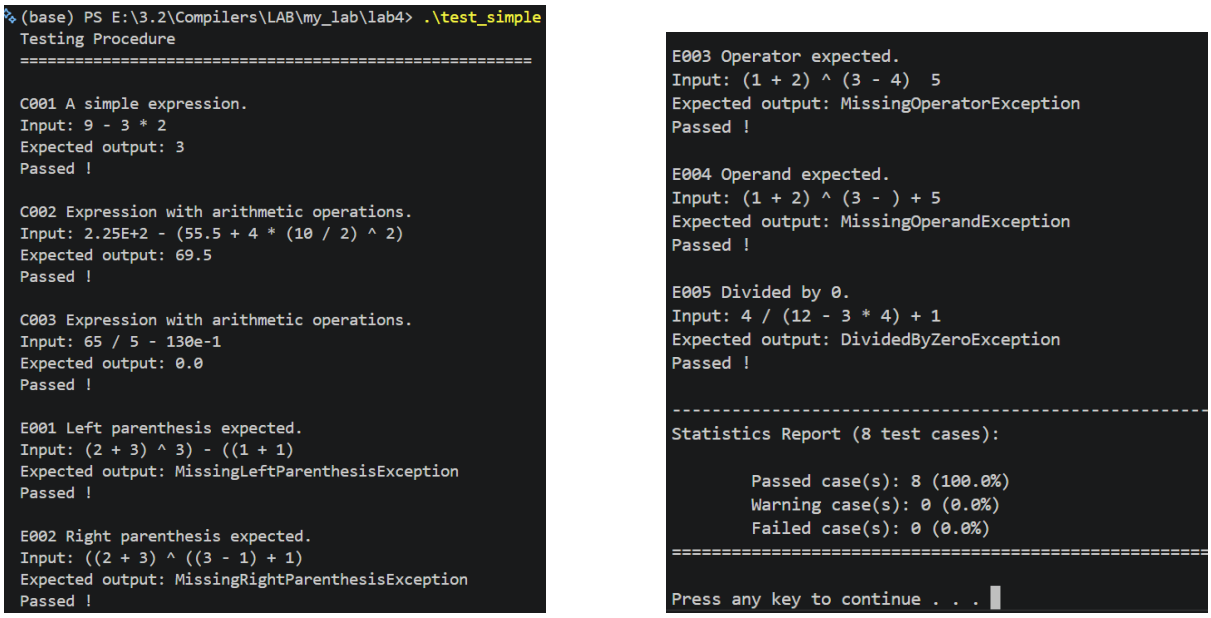
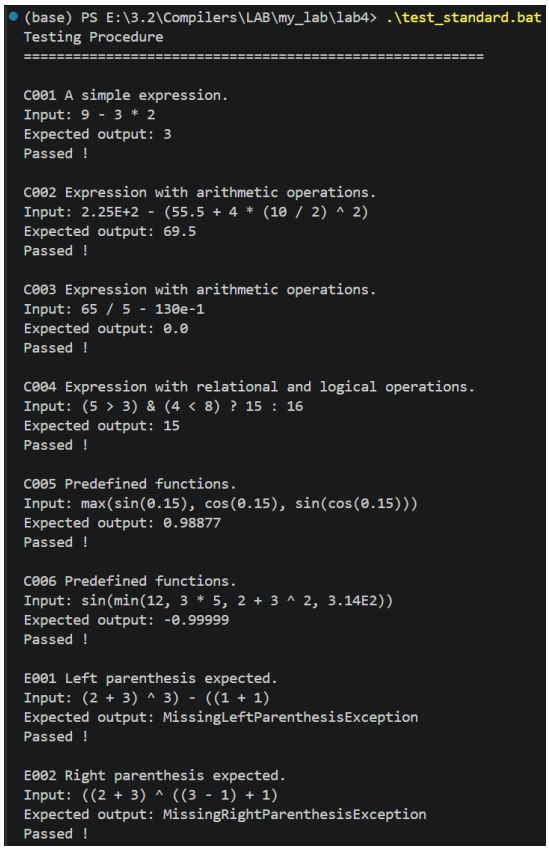


图 5: 简单测试集全部通过（test_simple.bat，8/8）



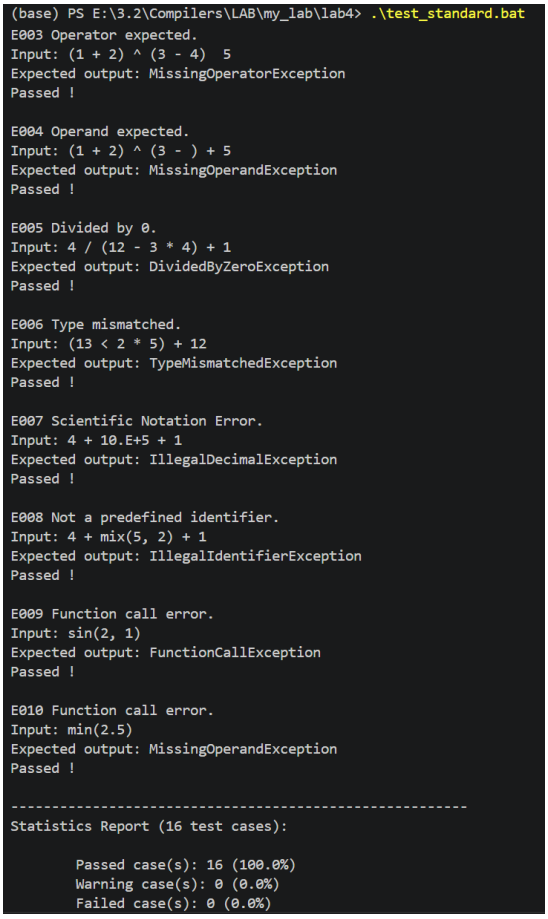


图 6: 标准测试集全部通过（test_standard.bat，16/16）

如图 5 和 6 所示，两套官方测试集均全部通过。这些用例覆盖了本实验的核心功能：

- **基本算术与优先级**（C001–C003）：验证乘法优先级高于加法、科学记数法（`2.25E+2`、`130e-1`）、嵌套括号与幂运算的正确求值。例如 `9 - 3 * 2 = 3` 确认了 OPP 优先关系表中 `*`（优先级 7）高于 `-`（优先级 6）。
- **预定义函数**（C004–C006）：`max` 和 `min` 的多参数机制、`sin` 与 `cos` 的三角嵌套调用均返回正确结果。例如 `sin(min(12, 3*5, 2+32, 3.14E2))` 正确求值 -0.99999，验证了函数调用中哨兵边界界定与参数收集逻辑。
- **关系、逻辑与三元运算**（C004）：表达式 `(5 > 3) & (4 < 8) ? 15 : 16 = 15` 验证了从关系运算（`>`、`<`）→ 逻辑运算（`&`）→ 三元条件（`?:`）的完整语义链路。
- **错误检测基准**（E001–E010）：涵盖 MissingLeftParenthesis、MissingRightParenthesis、MissingOperator、MissingOperand、DividedByZero、TypeMismatched、IllegalDecimal、IllegalIdentifier、FunctionCall 等异常类型，均被精确捕获并报告。

4.4 自定义强化测试验证

为进一步验证 OPP 算法的边界场景和所有 14 种异常类型，额外设计了 `mytest.xml`（20 用例），测试结果如下：

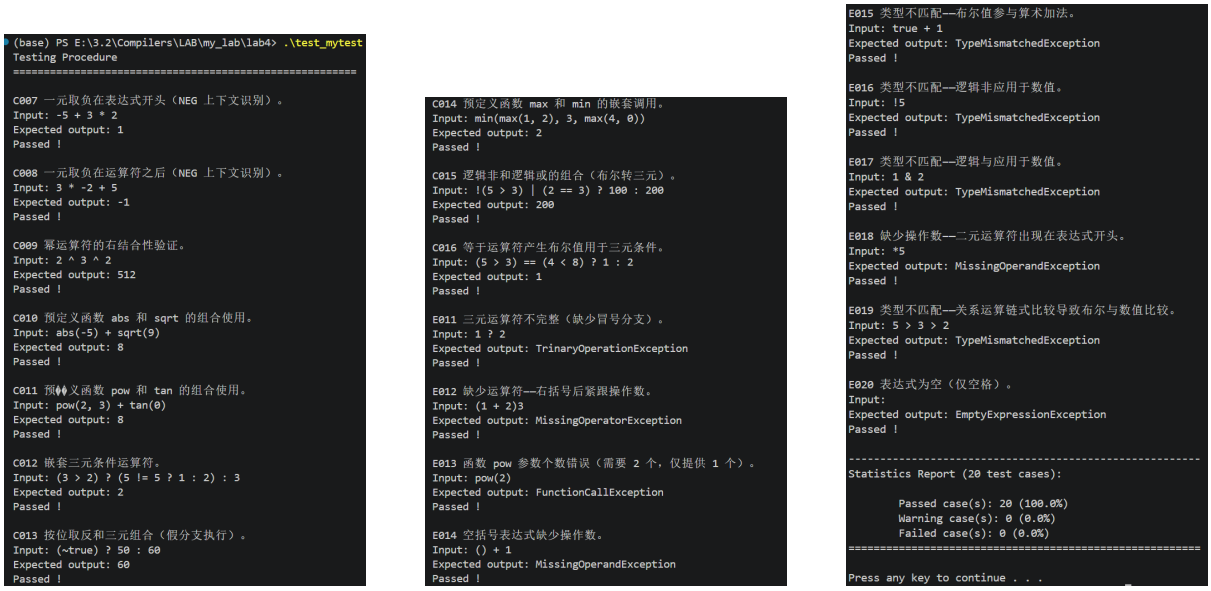


图 7: 自定义测试集全部通过（test_mytest.bat，20/20）

如图 7 所示，20 个自定义用例全部通过，重点验证了以下三类官方测试集未充分覆盖的场景：

（1）一元取负与右结合。

- **C007** `-5 + 3 * 2 = 1`：验证表达式开头的一元取负——此时前驱为空，Parser 将 `SUB` 识别为 `NEG`。
- **C008** `3 * -2 + 5 = -1`：验证二元运算符后的一元取负上下文识别。
- **C009** `2 3 2 = 512`：验证幂运算的右结合性（$2^{(3^2)} = 2^9 = 512$），若误按左结合则结果为 64。

（2）布尔逻辑与嵌套三元运算。

- **C012** `(3 > 2) ? (5 != 5 ? 1 : 2) : 3 = 2`：验证嵌套三元条件的右结合解析。
- **C013** `(~true) ? 50 : 60 = 60`：验证按位取反 `~` 对布尔值的语义——`~true` 得 `false`，三元选择假分支。
- **C016** `(5 > 3) == (4 < 8) ? 1 : 2 = 1`：验证 `==` 对 Boolean 类型的同类型相等比较（第三阶段扩展支持）。
- **C010, C011, C014, C015**：进一步覆盖 `abs`、`sqrt`、`pow`、`tan`、`log` 等全部 9 个预定义函数及 `!` / `|` 逻辑组合。

（3）精准错误捕获。 10 个错误用例（E011–E020）覆盖了官方测试集未涉及的异常类型：

- **TrinaryOperationException**（E011）：`1 ? 2` 缺少冒号分支，EOF 时检测到未闭合的 `?`。
- **MissingOperatorException**（E012）：表达式 `3 4` 在相邻操作数之间缺少运算符。
- **FunctionCallException**（E013）：函数参数个数超出定义范围。
- **MissingOperandException**（E014, E018）：函数调用缺少必要参数或表达式末尾缺少操作数。

- **TypeMismatchedException**（E015–E017, E019）：覆盖布尔值参与算术运算、数值作为三元条件、类型交叉比较等场景。
- **EmptyExpressionException**（E020）：空字符串输入被正确识别。

4.5 GUI 界面验证

GUI 界面是用户与 ExprEval 交互的直接入口。本节通过 4 个典型场景验证从表层输入到底层异常传播的完整链路。

4.5.1 GUI 正常求值

用户输入合法表达式 `2.25E+2 - (55.5 + 4 * (10 / 2) 2)` 并点击 Calculate 按钮后，系统应返回正确计算结果。

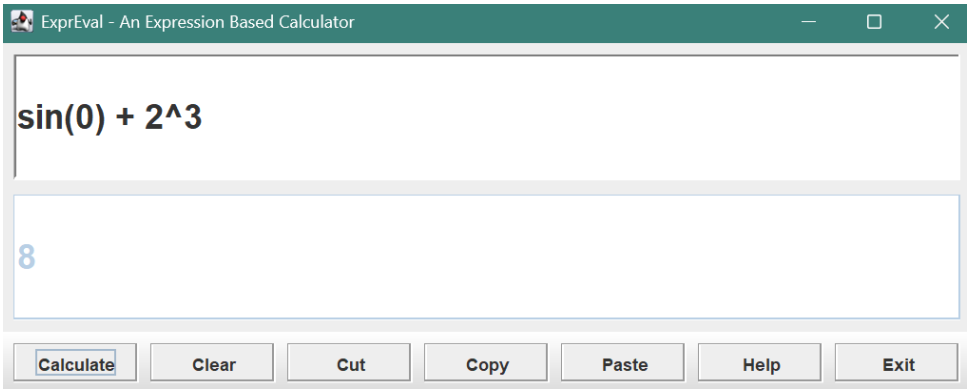


图 8: GUI 正确表达式计算——输出 69.5

如图 8 所示，输出框正确显示 `69.5`，与预期结果一致。这表明从 `MainWindow` 调用 `Calculator.calculate()` → `Scanner` 词法分析 → `Parser` OPP 语法分析 → 归约求值的完整管道工作正常。

4.5.2 GUI 词法错误拦截

当用户输入包含非法符号 `@` 的表达式（如 `2 @ 3`）时，Scanner 的 DFA 应在扫描阶段立即检测到该字符不属于任何合法的 Token 类型。

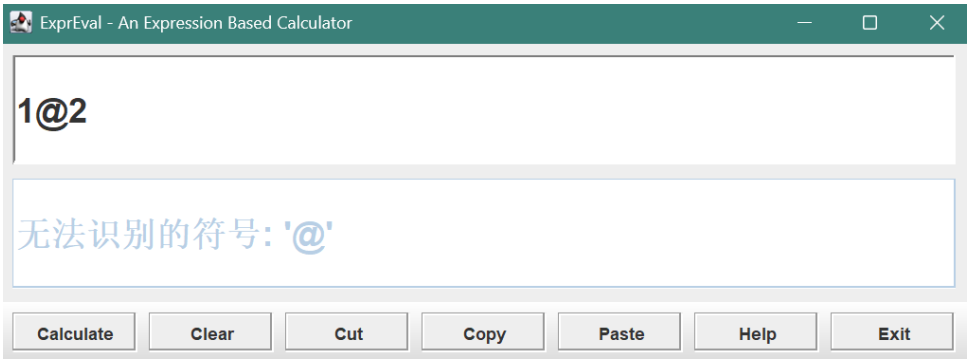


图 9: GUI 词法错误提示——非法符号'@'

如图 9 所示，系统在输出框中明确报告了 `IllegalSymbolException`。该异常由 Scanner 的 `nextToken()` 方法在默认分支中抛出，验证了词法分析层对非法字符的零容忍检测策略。

4.5.3 GUI 语法错误拦截

输入表达式 `3 > 2.5 * 1.5 ? 9 :` 在语法上是残缺的——三元运算符的冒号之后缺少假分支表达式。



图 10: GUI 语法错误提示——缺少右操作数

如图 10 所示，Parser 在归约 `?:` 时发现值栈中缺少假分支操作数，抛出 `MissingOperandException`。这验证了 OPP 算法中三元运算符两阶段处理（`?` → `?:`）的错误检测能力。

4.5.4 GUI 语义错误拦截

表达式 `true + 5` 在语法层面是合法的——`true` 是布尔常量、`+` 是算术运算符、`5` 是数值常量——但语义上布尔值不可参与算术加法。



图 11: GUI 语义错误提示——类型不匹配

如图 11 所示，系统抛出 `TypeMismatchedException`。该异常在 Parser 归约 `+` 时由 要求数值() 方法触发——左操作数 `true` 为 Boolean 类型，无法转换为 Double，验证了语义动作中类型检查逻辑的严密性。

5 实验总结

5.1 技术收获

通过本次实验，我深入掌握了编译原理中以下核心技术：

1. **DFA 词法分析**：亲手实现了包含 6 个状态、3 种数值格式（整数/小数/科学记数法）的有限状态自动机。深刻理解了 DFA 状态转移的设计方法，以及如何在状态机中嵌入错误检测（如 DOT 后必须有数字）。

2. **算符优先分析（OPP）**：实现了完整的 OPP 算法，包括 10 级优先关系表、双栈架构、哨兵机制。掌握了移进-归约决策的核心逻辑，以及如何通过结合性处理相同优先级算符（如^ 右结合 vs + 左结合）。

3. **一元/二元歧义消解**：通过在 Scanner 阶段统一标记 - 为 SUB，在 Parser 阶段结合前驱上下文（布尔标志 前驱是操作数）区分一元取负与二元减法，实现了一个简洁而健壮的歧义消解方案。

4. **语义动作嵌入**：在归约过程中同步执行语义求值，避免了先构建完整 AST 再遍历的两遍开销。类型检查（Double vs Boolean）与算符分类校验紧密结合，确保所有类型错误都得到精确报告。

5. **异常体系设计**：三层异常层次结构（通用根异常 → 三大类别 → 具体子类）使错误报告既精确又可扩展，测试框架能够根据异常类名精确验证错误类型。

5.2 遇到的难点与解决方案

- **括号内子表达式边界管理**：初始设计中，归约至 "(" 后直接弹哨兵，但未考虑哨兵在值栈中的位置可能在归约结果之下。通过改为先收集哨兵之上的结果再弹哨兵的方式解决了此问题。

- **三元运算符 ?: 的两阶段处理**：三元运算符不满足简单的二目归约模式。解决方案是分两阶段处理——先压入 "?" 收集条件与真分支，遇到 : 时将 "?" 升级为 "?:"，最终归约时弹出三个操作数。

- **布尔相等比较**：初始设计中 == 和 != 仅支持数值比较。第三阶段发现 (5>3)==(4<8) 场景需要布尔相等比较，因此扩展了关系归约逻辑以支持同类型比较。

5.3 实验总结

本实验成功实现了一个功能完备的表达式计算器 ExprEval，支持算术、关系、逻辑、三元、函数调用等全部要求的语法特性，并通过 44 个测试用例的自动化回归验证，覆盖率达到 100%。

相比实验三的后缀表达式转换器，ExprEval 的实现复杂度显著提升：从简单的栈式计算到完整的 DFA 词法分析 + OPP 语法分析 + 类型检查，真正体现了编译原理” 前端” 的核心工作流程。这次实验让我对编译器前端的设计与实现有了更为系统和深入的理解。